Informe de Sistema distribuido para servicios de computación de alto rendimiento (HPC) – UPB-CIENTÍFICA

Andrés Felipe Pinzón Camacho, Sergio Alejandro Sanabria Moreno, Angie Lorena Villareal Vásquez, Valentina Téllez Salazar, Fernando Cancelado Figueroa y Daniel Felipe Aguilar Latorre

Universidad Pontificia Bolivariana Seccional Bucaramanga, COLOMBIA

Resumen. Este artículo presenta el diseño, la implementación y la validación de un sistema distribuido que dota a la microempresa UPB-CIENTÍFICA de servicios de computación de alto rendimiento (HPC) y de gestión colaborativa de archivos, alojado en el Centro de Computación Avanzada de la Universidad Pontificia Bolivariana seccional Bucaramanga. El sistema recibe código fuente y conjuntos de datos y los ejecuta en paralelo con MPI mediante un gestor del clúster en Java RMI, e integra los servicios complementarios de la Figura 1 del enunciado —almacenamiento distribuido de archivos, fotos y videos; sincronización de archivos con gRPC; compartición con permisos Unix; álbum de fotografías y streaming de video— bajo una autenticación centralizada (servicio SOAP sobre un directorio LDAP), cifrado TLS y monitoreo con Prometheus y Grafana. Se describen la arquitectura orientada a servicios adoptada, su despliegue en tres servidores virtuales y en un clúster de cuatro máquinas virtuales con OpenMPI, la justificación de las tecnologías y la validación mediante pruebas de extremo a extremo de la ejecución distribuida, la integración de los clientes web, de escritorio y móvil, la precisión de la telemetría y los esquemas de seguridad.

Palabras clave: sistemas distribuidos, arquitectura orientada a servicios, computación de alto rendimiento, computación en malla, RMI, gRPC, SOAP, MPI.

1 Introducción

La computación de alto rendimiento (HPC) se ha convertido en un recurso estratégico para la comunidad científica, al permitir el procesamiento de grandes volúmenes de datos y la ejecución de simulaciones complejas en tiempos razonables. Sin embargo, poner esta capacidad de cómputo al servicio de investigadores externos exige mucho más que un clúster de máquinas: requiere una infraestructura distribuida capaz de administrar el acceso, almacenar y sincronizar los datos de trabajo, compartir resultados de forma segura y monitorear continuamente su disponibilidad.

Este artículo presenta el sistema distribuido de la microempresa UPB-CIENTÍFICA, dedicada a ofrecer servicios de HPC a la comunidad científica mediante el procesamiento paralelo de código basado en MPI. Como parte del proyecto de aula de la asignatura Sistemas Distribuidos (periodo 2026-20) de la Universidad Pontificia Bolivariana seccional Bucaramanga, el sistema integra una infraestructura de cómputo en malla (Grid), capaz de sumar al clúster computadoras, dispositivos móviles y elementos de IoT tanto desde el centro de cómputo como de forma externa, con servicios complementarios de almacenamiento distribuido, sincronización, compartición, álbum de fotografías y streaming, todos alojados en el Centro de Computación Avanzada de la universidad y desarrollados exclusivamente con software libre. Los usuarios acceden a ellos desde clientes nativos para Windows, Linux y Android y desde una interfaz web disponible en cualquier navegador.

El documento se organiza de la siguiente manera: la sección 2 presenta el estado del arte que sustenta las decisiones tecnológicas; la sección 3 expone el marco conceptual; la sección 4 detalla los objetivos; la sección 5 describe la metodología; la sección 6 presenta la arquitectura, las interfaces de los servicios, el despliegue y la justificación de las tecnologías; la sección 7 reporta el estado de los componentes, la validación de extremo a extremo y el rendimiento del clúster con un trabajo k-means; la sección 8 presenta el cronograma, y la sección 9 cierra con la discusión y las conclusiones.

2 Estado del arte

A continuación, se presentan los ejes temáticos clave que sustentan el diseño técnico del proyecto, organizados según los componentes tecnológicos exigidos por la situación problemática de UPB-CIENTÍFICA: computación en malla, middleware de objetos distribuidos, protocolos de comunicación entre servicios, sincronización y compartición de archivos, streaming de video, autenticación centralizada, seguridad, monitoreo de infraestructura HPC, cómputo paralelo con MPI y tolerancia a fallos.

2.1 Computación en malla y su comparación con la nube

Adepoju y Ologunagba (2025) presentan una revisión sistemática que compara la evolución de la computación en malla frente a las infraestructuras modernas de computación en la nube, examinando arquitecturas, aplicaciones y tendencias emergentes de ambos paradigmas. Los autores destacan que, si bien la nube ofrece elasticidad y facilidad de aprovisionamiento, la computación en malla conserva ventajas en escenarios donde se requiere integrar recursos heterogéneos y geográficamente distribuidos — como computadores, dispositivos móviles y equipos de laboratorio— bajo un esquema de propiedad compartida entre instituciones, en lugar de depender de un proveedor comercial único. En un sentido similar, Sungkar y Kogoya (2020) describen la computación Grid como un tipo de sistema paralelo y distribuido que permite compartir, seleccionar y agregar recursos autónomos dispersos geográficamente en tiempo de ejecución, según su disponibilidad, capacidad y los requisitos de calidad de servicio del usuario. Para UPB-CIENTÍFICA estos hallazgos son directamente aplicables: la exigencia de integrar computadoras, dispositivos móviles e IoT al clúster HPC, tanto interna como externamente, corresponde al modelo de computación en malla descrito en ambos estudios, y no a un esquema de nube pública, que además está descartado por las restricciones presupuestales del proyecto.

2.2 Middleware de objetos distribuidos: RMI y CORBA

La Figura 1 del enunciado asigna Java RMI al servidor del clúster (“RMI Server Java Cluster”), por lo que el gestor que recibe, planifica y coordina los trabajos MPI exige comprender las fortalezas y limitaciones de RMI frente a alternativas como CORBA. Munoz y Zalewski (2001) realizan una comparación arquitectónica y de desempeño entre RMI y CORBA sobre plataformas Solaris y Windows NT, midiendo el tiempo de respuesta ante invocaciones de métodos remotos con distinto número de parámetros y bajo carga de múltiples clientes concurrentes. Juric (2000) complementa este análisis con una comparación cualitativa y cuantitativa centrada en la facilidad de desarrollo, la madurez y el desempeño de RMI frente a CORBA en aplicaciones basadas en Java, y concluye que RMI resulta más sencillo de integrar en entornos homogéneos Java, mientras que CORBA maneja mejor la interoperabilidad multilenguaje con mayor sobrecarga. Estos hallazgos justifican, dentro de la arquitectura de UPB-CIENTÍFICA, el uso de Java RMI en el gestor del clúster HPC: sus componentes (gestor, fachada de trabajos y ejecutor en el nodo

maestro) están escritos en Java y no requieren interoperabilidad multilenguaje en ese tramo; el resto de clientes accede a él a través de una fachada REST.

2.3 Protocolos de comunicación entre servicios: SOAP, REST y gRPC

El sistema integra los estilos de comunicación que muestra la Figura 1 del enunciado: SOAP (servidor PHP de usuarios y autenticación), un servidor web basado en JavaScript, Java RMI (gestor del clúster) y gRPC (servidor Go de sincronización de archivos), además de APIs REST para los servicios de implementación. Castillo, Bernier, Arenas, Merelo y García-Sánchez (2011) comparan experimentalmente SOAP y REST implementando un algoritmo genético maestro-esclavo sobre ambos protocolos: SOAP resulta conceptualmente más complejo y pesado, pero ofrece un contrato de interfaz más formal (WSDL), adecuado para un servicio de identidad que consumen todos los demás; REST resulta más liviano para escenarios de alta frecuencia de solicitudes. Por su parte, la literatura técnica reciente coincide en que gRPC, al basarse en HTTP/2 y en la serialización binaria de Protocol Buffers, ofrece menor latencia y mayor throughput que SOAP o REST, y es especialmente adecuado para transferencias en flujo (streaming) bidireccional, lo que explica su elección para el servicio de sincronización de archivos, que transfiere árboles de directorios y contenido binario entre los clientes y el servidor.

2.4 Sincronización y compartición de archivos distribuidos

El servicio File Sync debe crear en el servidor una copia síncrona de seguridad de un directorio local del cliente, de forma programada en el escritorio y al alcanzar la red Wi-Fi local en el móvil, manteniendo consistencia entre todas las copias. Mehdi y Ragab-Hassen (2016) presentan una encuesta de tecnologías de sincronización de archivos, tanto centralizadas en la nube (Dropbox, Google Drive) como basadas en P2P, y discuten sus fortalezas y debilidades en términos de ancho de banda, latencia y consistencia; destacan que los protocolos que solo transmiten los cambios reducen significativamente el consumo de red, principio aplicable al servicio de copia síncrona exigido por el proyecto. Complementariamente, el estudio cualitativo de Capra (2014), basado en entrevistas a usuarios universitarios, identifica limitaciones prácticas de la sincronización automática, como la imposibilidad de sincronizar carpetas parcialmente o la necesidad de procesos manuales de respaldo, relevantes para el diseño del Home parametrizado que exige UPB-CIENTÍFICA para cada cliente.

2.5 Arquitecturas de streaming de video adaptativo

El servicio de Streaming debe ofrecer reproducción fluida, control de acceso y gestión de permisos para los videos que los científicos comparten en presentaciones empresariales. Natali y Merani (2015) proponen una arquitectura de streaming adaptativo (P2P-DASH) que combina Dynamic Adaptive Streaming over HTTP con redes peer-to-peer y una estrategia de control de tasa distribuida que ajusta la calidad del video según las condiciones de la red. En una línea similar, Waheed et al. (2021) presentan una arquitectura eficiente para streaming en tasa de bits adaptativa, y demuestran que el consumo de CPU y energía puede optimizarse ajustando el tamaño de segmento sin sacrificar la calidad percibida. Ambos estudios orientan el diseño del servicio de streaming, que debe equilibrar la calidad de reproducción y el consumo de recursos del servidor, dado que la infraestructura se aloja íntegramente en el Centro de Computación Avanzada sin redes de distribución de contenido externas.

2.6 Autenticación centralizada y directorios activos

UPB-CIENTÍFICA exige que los perfiles de usuario —autenticación, autorización, mapa de servicios y árboles de archivos— sean administrados por un servicio de directorio activo. El trabajo clásico de Woo y Lam (1992) sienta las bases teóricas de la autenticación en sistemas distribuidos, formalizando los requisitos de un protocolo seguro frente a ataques de intermediario y repetición, principios vigentes en los protocolos que usan servicios de directorio como LDAP y Active Directory. La literatura técnica reciente coincide en que estas tecnologías siguen siendo fundacionales para la gestión centralizada de identidades. Esto sustenta la decisión de emplear un directorio OpenLDAP, expuesto mediante el servidor SOAP en PHP de la Figura 1, como fuente única de verdad para los perfiles compartidos entre los servicios del sistema (File Sync, Shared File, Photo Album, Streaming y MPI Jobs).

2.7 Seguridad en sistemas distribuidos

El proyecto exige cifrado asimétrico GPG para las copias de seguridad, autenticación de los clientes basada en token o múltiple factor, y un análisis de vulnerabilidades de la aplicación web en su ambiente de producción con los niveles de riesgo asumidos claramente documentados. Priyadarshini (2022) divide el problema de la seguridad distribuida en dos frentes: la seguridad de la comunicación entre usuarios y procesos —autenticación, integridad y cifrado— y el control de los permisos de acceso a los recursos. Babu, Abraham y Borasia (2013) revisan la criptografía de clave simétrica y enumeran los pilares de la seguridad distribuida: autenticación, confidencialidad, autorización, integridad, disponibilidad y trazabilidad. Estos marcos son aplicables al diseño de seguridad de UPB-CIENTÍFICA, en el que conviven el cifrado asimétrico de las copias (confidencialidad de datos en reposo) con tokens de sesión firmados (verificación de identidad) y permisos Unix sobre los archivos (autorización).

2.8 Monitoreo de clústeres e infraestructuras HPC

El módulo de monitoreo debe hacer seguimiento al servicio HPC y a los demás componentes de la infraestructura, con estadísticas en tiempo real de ocupación de recursos (CPU, memoria, almacenamiento), carga de trabajos, tiempos de ejecución y disponibilidad de servicios, accesible vía web, con reportes históricos y alertas configurables. Kunz (2022) implementa una solución de monitoreo de trabajos HPC basada en SLURM, Prometheus y Grafana, y evalúa marcos como PIKA y ClusterCockpit. La documentación técnica de referencia coincide en que este patrón —exportadores de métricas por nodo que alimentan una base de series temporales, con paneles de visualización y alertas configurables— es el estándar de facto para clústeres on-premise, lo que sustenta la elección de Prometheus y Grafana, ambos de software libre.

2.9 MPI y paradigmas de cómputo paralelo

AlHafez y Kurdi (2025) comparan MPI, OpenMP y CUDA y muestran que MPI sobresale en entornos de memoria distribuida, con una escalabilidad casi lineal en aplicaciones intensivas en comunicación entre nodos, mientras que OpenMP resulta más adecuado para el paralelismo de memoria compartida dentro de un nodo. Esto confirma que MPI es la tecnología correcta para coordinar el procesamiento entre los nodos del clúster de la infraestructura Grid, reservando los paradigmas de memoria compartida para la optimización interna de cada nodo.

2.10 Tolerancia a fallos y replicación

Aunque el enunciado no detalla mecanismos de replicación, la carta descriptiva del curso (UPB, 2020) incluye la tolerancia a fallos y la alta disponibilidad como aspectos transversales a cualquier sistema distribuido. El trabajo tutorial de Guerraoui y Schiper (1996) define la linealizabilidad como criterio de corrección para servicios replicados y presenta la replicación primario-respaldo y la replicación activa. Estos conceptos son pertinentes en dos frentes: el repositorio distribuido de archivos, fotos y videos podría beneficiarse de réplicas para tolerar la caída de un nodo, y el clúster HPC, al ejecutar trabajos de larga duración, requiere mecanismos de recuperación ante fallos (reintento de trabajos, reasignación de nodos caídos).

3 Marco conceptual

3.1 Sistema distribuido

Conjunto de nodos autónomos que se comunican por red y se presentan ante el usuario como un único sistema coherente, coordinando su funcionamiento sin depender de un reloj o una memoria compartida.

3.2 Computación en malla (Grid computing)

Modelo de sistema distribuido en el que se comparten, seleccionan y agregan recursos autónomos y geográficamente dispersos —computadoras, dispositivos móviles y elementos de IoT— según su disponibilidad, capacidad y los requisitos de calidad de servicio del usuario, sin depender de un proveedor comercial único.

3.3 Computación de alto rendimiento (HPC)

Subconjunto de la computación distribuida orientado a maximizar la capacidad de procesamiento mediante clústeres de múltiples nodos, empleados para ejecutar trabajos científicos, de ingeniería o de análisis de datos a gran escala.

3.4 MPI

Interfaz de paso de mensajes (Message Passing Interface) que coordina la ejecución de un mismo programa entre múltiples procesos que no comparten memoria.

3.5 Middleware de objetos distribuidos

Capas de software (RMI en Java, CORBA de forma multilenguaje) que permiten invocar métodos de objetos ubicados en procesos remotos como si fueran locales.

3.6 Servicios web y APIs (SOAP, REST y gRPC)

Estilos de comunicación sobre los que se apoya el proyecto: SOAP, basado en XML y con contrato formal (WSDL); REST, orientado a recursos sobre HTTP; y gRPC, basado en HTTP/2 y Protocol Buffers.

3.7 Sincronización de archivos

Mecanismo por el cual los cambios en el árbol de directorios de un dispositivo cliente se propagan al repositorio central y a las demás copias, manteniendo una versión del recurso en el servidor.

3.8 Directorio activo

Servicio centralizado (LDAP / Active Directory) que administra los perfiles de usuario: autenticación, autorización, mapa de servicios y árboles de archivos.

3.9 Seguridad informática distribuida

Conjunto de mecanismos —cifrado, autenticación, integridad, disponibilidad y trazabilidad— que protegen la comunicación entre procesos y el acceso a los recursos.

3.10 Monitoreo de infraestructura

Recolección continua de métricas de uso (CPU, memoria, almacenamiento, carga de trabajos, tiempos de ejecución y disponibilidad) mediante exportadores por nodo, una base de series temporales y paneles de visualización y alertas.

3.11 Arquitectura orientada a servicios (SOA)

Estilo arquitectónico en el que el sistema se compone de servicios de grano grueso, autónomos y posiblemente heterogéneos, que se comunican exclusivamente a través de contratos de interfaz estables (WSDL, IDL, Protocol Buffers o especificaciones REST). A diferencia de los microservicios, no persigue el despliegue independiente por contenedor ni la consistencia eventual entre réplicas de datos, sino la interoperabilidad entre tecnologías distintas mediante contratos.

4 Objetivos

4.1 Objetivo general

Desarrollar un sistema distribuido que proporcione servicios de computación de alto rendimiento (HPC) y gestión colaborativa de archivos, mediante una infraestructura en malla alojada en el Centro de Computación Avanzada de la Universidad Pontificia Bolivariana seccional Bucaramanga, para permitir a la microempresa UPB-CIENTÍFICA y a la comunidad científica ejecutar aplicaciones de procesamiento intensivo de manera segura y escalable, así como administrar, compartir y presentar recursos y datos científicos de forma centralizada.

4.2 Objetivos específicos

1. Documentar los requerimientos funcionales y no funcionales de la computación en malla, la computación de alto rendimiento y los paradigmas de comunicación distribuida pertinentes, con el fin de definir la arquitectura general del entorno de administración y ejecución de trabajos paralelos en MPI.
2. Desarrollar el entorno de administración y ejecución de código en paralelo con MPI sobre una infraestructura en malla (Grid), con un gestor del clúster en Java RMI que reciba el código fuente y los conjuntos de datos, planifique los trabajos y admita la integración de equipos internos y externos.
3. Desarrollar el servicio de sincronización de archivos en gRPC que cree en el servidor una copia síncrona de un directorio local, de forma programada en los equipos de escritorio y al detectar la red local en los dispositivos móviles, manteniendo la consistencia entre todas las copias.

4. Desarrollar el repositorio distribuido de archivos, fotos y videos y el servicio de compartición de archivos, con una interfaz de carga al Home del usuario, respetando el modelo de permisos Unix sobre un espacio parametrizado y con cuota máxima por usuario.
5. Desarrollar el servicio de álbum de fotografías, que funcione sobre la web y permita buscar, ordenar, presentar y compartir imágenes interna y externamente, y el servicio de streaming de video, con funciones de ordenamiento, reproducción fluida, control de acceso y gestión de permisos.
6. Integrar los componentes del sistema mediante las APIs especificadas para el proyecto (SOAP, JavaScript, Java RMI, gRPC y REST), garantizando la interoperabilidad entre los servicios desplegados en los distintos servidores.
7. Desarrollar un servicio de directorio activo para la administración centralizada de los perfiles de usuario, incluyendo autenticación, autorización, mapa de servicios y árboles de archivos.
8. Implementar mecanismos de seguridad informática mediante cifrado asimétrico GPG para las copias de seguridad, autenticación de los clientes basada en token y análisis de vulnerabilidades de la aplicación web en su ambiente de producción, con los niveles de riesgo asumidos documentados.
9. Implementar un módulo de monitoreo integral, accesible vía web, que reporte en tiempo real la ocupación de recursos, la carga de trabajos, los tiempos de ejecución y la disponibilidad de los servicios, con históricos y alertas configurables.
10. Validar el sistema mediante pruebas de extremo a extremo para verificar la correcta ejecución distribuida de trabajos MPI, la integración funcional de los clientes web, de escritorio y móvil, la precisión de la telemetría transversal y el cumplimiento de los esquemas de seguridad.

5 Metodología

El proyecto adopta un modelo de desarrollo iterativo e incremental (Sommerville, 2011), en el que el ciclo de vida del software —análisis, diseño, implementación, pruebas e integración— se recorre completo en cada iteración. Esta elección responde a un proyecto de infraestructura distribuida con múltiples servicios desarrollados en paralelo por un equipo de seis integrantes, que entrega en cada iteración un incremento funcional y verificable, alineado con las entregas de la asignatura (bitácoras en las semanas 8 y 16, producto en la semana 14). Antes de escribir código se fijaron los contratos de cada servicio (WSDL, .proto, interfaces RMI y REST), de modo que cada subequipo pudiera avanzar contra simulaciones de los servicios que consume.

Cada iteración recorre el mismo ciclo de cinco pasos. Primero, los requisitos: 26 requerimientos funcionales especificados con el estándar IEEE 830 y priorizados con el método MoSCoW. Segundo, el diseño de contratos: un contrato por servicio (WSDL, .proto, interfaz RMI o REST) que fija operaciones, datos y errores antes de escribir código. Tercero, la implementación, en la que cada subequipo programa contra simulaciones de los servicios que consume. Cuarto, las pruebas y la integración, apoyadas en un validador de tokens intercambiable entre un modo simulado y el servicio SOAP real. Quinto, el incremento, que se suma al sistema y abre la siguiente iteración.

El desarrollo se organiza en cuatro iteraciones: la Iteración 1 cubre la infraestructura Grid base, el directorio activo y la configuración del clúster HPC; la Iteración 2, los servicios File Sync (gRPC) y Shared File sobre el repositorio distribuido; la Iteración 3, los servicios Photo Album y Streaming; y la Iteración 4, el servicio MPI Jobs (gestor del clúster en Java RMI) y el módulo de monitoreo. Cada iteración termina con una revisión del registro de riesgos y con la integración incremental de los servicios construidos sobre la versión anterior del sistema.

La verificación siguió el mismo enfoque incremental. Cada servicio se probó primero contra dobles de los servicios que consume —en particular, un validador de tokens intercambiable entre un modo simulado y el servicio SOAP real, patrón Strategy— y luego en integración. Para la validación final se construyó un conjunto de pruebas de extremo a extremo que se ejecuta contra el sistema desplegado, sin dobles: cada prueba usa las APIs reales, con usuarios del directorio, y registra sus resultados en formato JSON y JUnit como evidencia reproducible. El despliegue se realizó sin contenedores, como exige el enunciado: cada servicio corre como unidad de systemd con un usuario sin privilegios, sobre paquetes de los repositorios oficiales del sistema operativo.

6 Arquitectura y tecnologías

6.1 Estilo arquitectónico

El sistema adopta una arquitectura orientada a servicios: siete servicios de grano grueso, escritos en las tecnologías que impone la Figura 1 del enunciado, que solo se comunican mediante contratos (Fig. 1). Cada servicio está organizado en capas —presentación, dominio y datos— con un único punto de composición de dependencias, y traduce sus errores de dominio a respuestas del protocolo en un solo lugar. Cuatro reglas transversales gobiernan la integración:

- **Una sola puerta para el navegador.** La interfaz web solo habla con el portal (node-web), que reenvía cada petición /api al servicio correspondiente sin interpretarla ni validarla. Los clientes nativos consumen los contratos de cada servicio directamente.
- **Identidad común, autorización en cada servicio.** El usuario inicia sesión una vez contra el servicio de autenticación; cada servicio valida el token y consulta el mapa de servicios del perfil. El identificador del usuario se obtiene siempre del token validado, nunca del cuerpo del mensaje.
- **Una única fuente de verdad para los archivos.** Distributed Storage es el único servicio que guarda bytes y el único que aplica cuotas y permisos; los demás (sincronización, medios, HPC) leen y escriben a través de su API con el token del usuario.
- **Ninguna base de datos compartida.** Cada servicio guarda en su propia base PostgreSQL solo sus metadatos e índices; las referencias entre almacenes las valida la API del servicio dueño del dato.

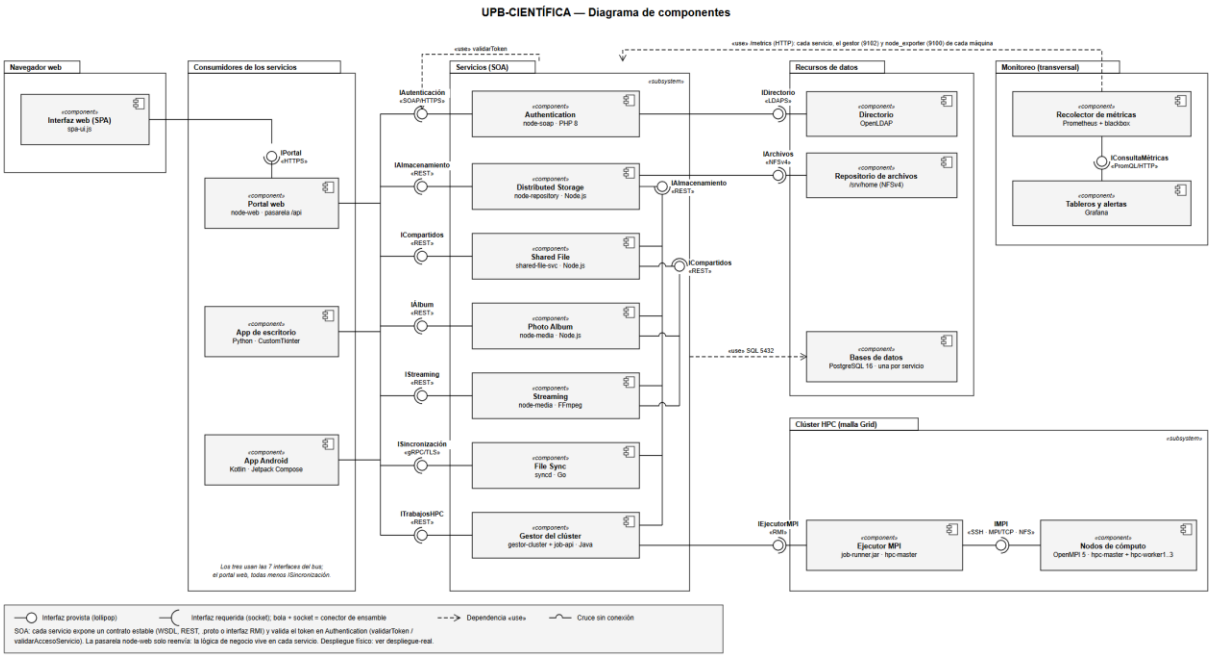


Fig. 1. Diagrama de componentes (UML 2.5): interfaces provistas y requeridas de cada servicio.

6.2 Interfaces de los servicios

Cada servicio expone su funcionalidad únicamente a través de interfaces con contrato explícito. La Fig. 1 las agrupa en cuatro dominios —acceso e identidad, archivos, medios y monitoreo, y cómputo HPC— y la Tabla 1 resume cada una con su proveedor, su protocolo y sus operaciones principales.

Tabla 1. Interfaces de los servicios: proveedor, protocolo y operaciones principales.

Interfaz · proveedor	Protocolo y puerto	Operaciones principales y reglas
IPortal · node-web	HTTPS 443 (Nginx → 8080)	Inicio y cierre de sesión (POST y DELETE /api/sesion, traducidos a SOAP); reenvío sin cambios de /api/storage, /api/shared-file, /api/album, /api/streaming, /api/jobs y /api/cluster; /monitoreo hacia Grafana; límite de intentos de inicio de sesión (429).
IAutenticación · Authentication	SOAP 1.1 / HTTPS 8081 (users.wsdl)	login(usuario, contrasena, origenCliente, codigoMfa) devuelve token, vigencia, uid, roles, grupos y mapa de servicios autorizados; logout; consulta de usuario, grupos y perfil; generación y validación del código MFA. Token firmado con HMAC-SHA256, vigencia de 8 h, revocable.
IValidación · Authentication	SOAP 1.1 / HTTPS 8081	validarToken y validarAccesoServicio(usuario, servicio). Todo servicio la invoca en cada petición: token inválido → 401; servicio fuera del perfil → 403.
IDirectorio · OpenLDAP	LDAPS 636 (solo local)	bind y búsqueda de usuarios y grupos del esquema upb-cientifica.
IAlmacenamiento · Distributed Storage	REST / HTTP 8084 (/storage)	Directorios, subida multipart, descarga por rangos (206), borrado, permisos efectivos y accesos por nodo, cuota. Única interfaz que lee o escribe bytes; aplica los permisos estilo Unix y la cuota.
IArchivos · Shared File	REST / HTTP 8085	Compartir archivos y carpetas con usuarios o grupos (LECTURA, LECTURA_ESCRITURA, LECTURA_ESCRITURA_COMPARTIR), cambiar el nivel, revocar, listar lo compartido conmigo y por mí. Decide quién accede; Storage aplica el permiso sobre los bytes.

Interfaz · proveedor	Protocolo y puerto	Operaciones principales y reglas
ISincronización · File Sync	gRPC sobre TLS 9090 (filesync.proto)	IniciarSync, SubirArchivo y DescargarArchivo por flujos de bloques, EstadoSync y EliminarArchivo. Tras comparar SHA-256 responde por archivo IGUAL, SUBIR, BAJAR, BORRAR, CONFLICTO o BORRAR_SERVIDOR; el token viaja en los metadatos.
IAlbum · Photo Album	REST / HTTP 8083 (/album)	Listar, buscar y ordenar fotos; miniatura y contenido; indexar el Home; crear álbumes y añadir o quitar fotos. El índice se deriva de Storage y de lo compartido en Shared File.
IStreaming · Streaming	REST / HTTP 8083 (/streaming)	Catálogo y búsqueda, miniatura, sesión de reproducción y contenido por rangos (206). Reenvía la cabecera Range a Storage, de modo que el video no se descarga completo.
ITrabajos · job-api	REST / HTTP 8082	Enviar trabajo (código, dataset, procesos, tiempo máximo, prioridad y parámetros); consultar estado, resultado y registros; cancelar, cambiar prioridad y reintentar; estado del clúster y registro de nodos. Fachada que traduce cada llamada a RMI.
ServicioHpc · gestor-cluster	Java RMI 2099 / 2100	Las mismas operaciones como métodos remotos que reciben el token, con excepciones tipadas en el contrato. Planifica con una cola de prioridad con envejecimiento y asigna los procesos según la carga de los nodos.
EjecutorMpi · job-runner (hpc-master)	Java RMI 2099 / 2100	iniciar, consultar, registros, resultados, cancelar, limpiar y nodosSinRespuesta. Compila con mpicc y lanza mpirun como mpiuser, con los argumentos en lista y sin intérprete de órdenes.
IMPI · nodos de cómputo	SSH 22, MPI sobre TCP, NFSv4	mpirun -np N --hostfile sobre hpc-master y hpc-worker1..3; código, dataset y resultados en /srv/home, compartido por NFS.
ISQL · PostgreSQL 16	PostgreSQL 5432 (solo local)	Metadatos de cada servicio en su propia base (metadatos de Storage, catálogo de medios). Los bytes quedan en el Home de cada usuario.
IMetricas e IConsultaMetricas · Prometheus	HTTP /metrics; PromQL	Peticiones, errores y latencia por servicio; cola y tiempos HPC; CPU, memoria, disco y red por máquina. Prometheus recolecta cada 15 s y Grafana consulta con PromQL para tableros y alertas.

Dos interfaces atraviesan todos los dominios. IValidación es requerida por cada servicio de negocio, que valida el token en cada petición y nunca toma la identidad del usuario del cuerpo del mensaje. IAlmacenamiento es requerida por Shared File, File Sync, Photo Album, Streaming y el gestor del clúster, que la usa para leer el código fuente y el dataset de cada trabajo. Las aplicaciones de escritorio y Android consumen directamente las interfaces de negocio; el navegador, en cambio, solo conoce IPortal.

6.3 Despliegue

La universidad asignó tres servidores virtuales (VPS) con Ubuntu Server 24.04 en lugar de una máquina por servicio. Los siete nodos lógicos del diseño se agruparon por afinidad y conservan sus nombres como alias de red, de modo que ningún servicio depende de la máquina física en la que corre (Tabla 2, Fig. 2). Cada VPS tiene un cortafuegos que solo admite los puertos del diagrama; las bases de datos, Prometheus y Grafana escuchan solo en la interfaz local. Todos los canales que transportan credenciales o tokens usan TLS con certificados firmados por una autoridad certificadora interna del proyecto, que los clientes nativos traen incluida.

Tabla 2. Distribución de los servicios en la infraestructura del CCA.

Máquina	Nodos lógicos	Servicios y puertos
pasd_nodo1 (VPS)	node-web, node-soap, node-monitoreo	Nginx 443 → portal 8080; Authentication SOAP/HTTPS 8081 sobre Apache + PHP; OpenLDAP (LDAPS, local); Prometheus y Grafana (locales, publicados en /monitoreo)
pasd_nodo2 (VPS)	node-repository, node-sync	Distributed Storage 8084, Shared File 8085, File Sync gRPC/TLS 9090; PostgreSQL 16; Home de los usuarios
pasd_nodo3 (VPS)	node-media, node-rmi	Photo Album y Streaming 8083 con FFmpeg y PostgreSQL 16; gestor del clúster (RMI 2099/2100) y fachada REST job-api 8082
Clúster HPC (4 VMs Debian 13)	hpc-master, worker1–3	job-runner (RMI) y OpenMPI 5 en el maestro; /srv/home compartido por NFSv4; el maestro también ejecuta procesos

El clúster se montó con máquinas virtuales Debian 13 en VirtualBox, una por PC del laboratorio, conectadas en modo puente a la red de la universidad con la dirección que esta asigna. Se descartó aislar los equipos con una red propia, porque cada PC dispone de una sola conexión de red y se perdía el acceso al resto de la infraestructura.

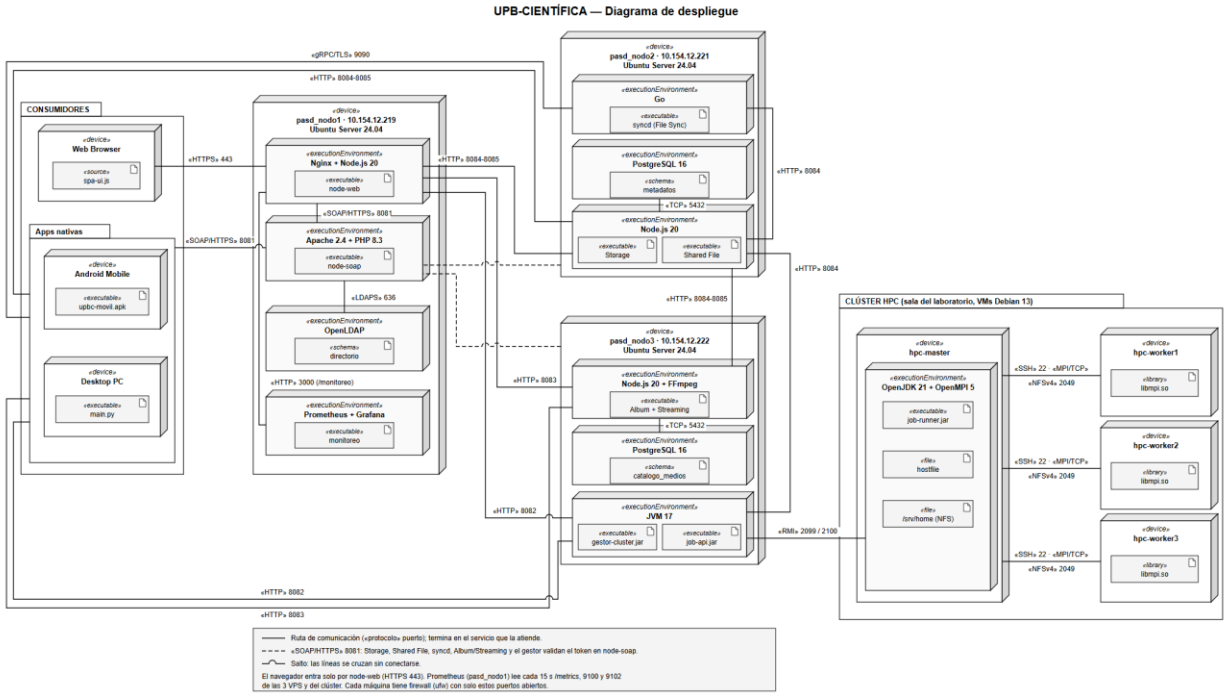


Fig. 2. Diagrama de despliegue del sistema: tres VPS del CCA y clúster del laboratorio, con protocolos y puertos.

6.4 Justificación de las tecnologías

Las tecnologías se eligieron con cinco criterios, en este orden: lo que impone el enunciado (Figura 1, software libre y sin contenedores ni servicios en la nube con costo); la adecuación al problema de cada servicio; la disponibilidad en los repositorios oficiales del sistema operativo, que sin contenedores garantiza actualizaciones de seguridad y la misma versión en todas las máquinas; el soporte a largo plazo, y la capacidad de publicar o consumir un contrato estándar (Tabla 3).

Tabla 3. Tecnologías empleadas y su justificación.

Tecnología	Uso	Justificación
PHP 8.3 + SOAP	Authentication	Impuesta por la Figura 1. El WSDL ofrece un contrato formal y versionado para el servicio del que dependen todos los demás; PHP trae soporte nativo de SOAP y LDAP.
OpenLDAP	Directorio	Estándar para perfiles administrados por un directorio: usuarios, grupos, mapa de servicios y directorio personal (homeDirectory).
Node.js 20 LTS + Express	Storage, Shared File, Album, Streaming, portal	«JavaScript» de la Figura 1. La E/S asíncrona y los streams sirven archivos y video por rangos sin cargarlos en memoria.
Go 1.22 + gRPC	File Sync	Impuesto por la Figura 1. Streams de bloques sobre HTTP/2 con un contrato .proto del que se genera el código de Go, Python y Kotlin; binario estático.
Java 17 LTS + RMI, Spring Boot 3	Gestor del clúster, job-runner, job-api	RMI impuesto por la Figura 1, con excepciones tipadas en el contrato; job-api es la fachada REST para clientes que no hablan RMI.
OpenMPI 5 + NFSv4	Clúster	MPI impuesto por el enunciado; OpenMPI es la implementación libre de referencia y la misma versión en todos los nodos; NFS comparte código y datos por la misma ruta.
PostgreSQL 16	Metadatos	Integridad referencial y transacciones para árbol de carpetas, versiones y permisos; se descartó SQLite por la escritura concurrente.
FFmpeg, GnuPG	Medios, respaldos	Miniaturas y metadatos de imágenes y videos; cifrado asimétrico OpenPGP exigido para las copias de seguridad.
Python 3.11 + CustomTkinter	Cliente de escritorio	Un solo código para Windows y Linux, con clientes maduros para SOAP, REST y gRPC.
Kotlin + Jetpack Compose, Retrofit, grpc-okhttp	Cliente Android	Plataforma nativa recomendada; habla cada protocolo real. Se descartaron React Native y Flutter por su soporte deficiente de SOAP y gRPC.
Nginx, systemd, ufw, CA interna	Operación y seguridad	Terminación TLS y cabeceras de seguridad; gestión de servicios sin contenedores; cortafuegos por máquina; TLS sin autoridad externa.
Prometheus, node_exporter, blackbox, Grafana	Monitoreo	Series temporales, métricas por máquina, sondas de disponibilidad, tableros y alertas, todo libre.
nmap, VirtualBox	Pruebas, clúster	Inventario de puertos y servicios expuestos para el análisis de vulnerabilidades; máquinas virtuales del clúster en los PC del laboratorio, sin contenedores.

6.5 Flujo de un trabajo HPC

La Fig. 3 muestra el envío de un trabajo desde el portal web. El navegador solo habla con node-web, que reenvía a job-api; este traduce la petición a RMI y el gestor valida el token y el perfil en el servicio de autenticación. Tras comprobar en Storage que el código y el conjunto de datos existen, el gestor encola el trabajo y responde de inmediato. Un planificador asíncrono, con prioridad y envejecimiento para evitar la inanición, asigna nodos según su carga y entrega el trabajo a job-runner en el nodo maestro, que lo compila con mpicc y lo lanza con mpirun en los nodos asignados. Al terminar, el gestor guarda los resultados en la carpeta «Resultados HPC» del Home del usuario. Los argumentos de mpirun se

construyen como lista, sin intérprete de comandos: ningún componente ejecuta órdenes enviadas por el usuario.

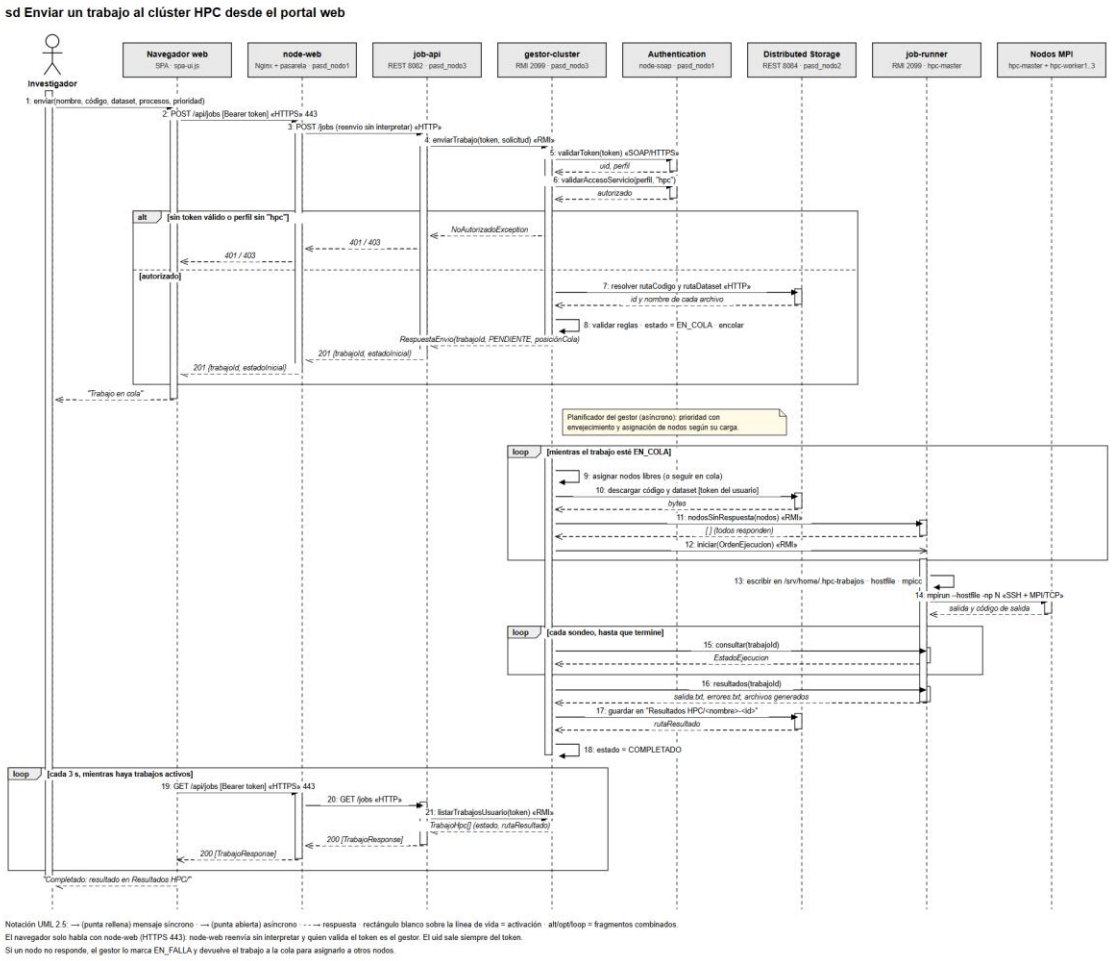


Fig. 3. Diagrama de secuencia del envío de un trabajo al clúster desde el portal web.

7 Resultados y validación

7.1 Estado de los componentes

A la semana 15 del periodo 2026-20 todos los servicios están desplegados en la infraestructura del CCA y los tres clientes operan contra ella. La Tabla 4 resume el estado de cada componente frente a los requisitos del enunciado.

Tabla 4. Estado de los componentes del sistema.

Componente (Figura 1)	Tecnología	Estado	Evidencia
Authentication / Users (directorio activo)	PHP 8.3 + SOAP + OpenLDAP	Desplegado (pasd_nodo1)	Token firmado con HMAC-SHA256, con vigencia y revocación; mapa de servicios por perfil; LDAPS solo local
Web Server JavaScript (portal)	Node.js 20 + Nginx	Desplegado (pasd_nodo1)	Pasarela única del navegador, sin lógica; TLS, HSTS, CSP y límite de intentos de inicio de sesión

Componente (Figura 1)	Tecnología	Estado	Evidencia
Repository (Distributed Storage)	Node.js + PostgreSQL 16	Desplegado (pasd_nodo2)	Home con cuota; permisos Unix y accesos heredables; versionado; descargas parciales
Shared File	Node.js + PostgreSQL 16	Desplegado (pasd_nodo2)	Compartición con usuarios y grupos, caducidad, revocación inmediata y herencia en carpetas
Photo Album y Streaming	Node.js + FFmpeg + PostgreSQL 16	Desplegado (pasd_nodo3); compartir con personas externas pendiente	Índice derivado de Storage, miniaturas, búsqueda y orden; reproducción por rangos; contenido compartido
File Sync (gRPC Server Go Sync)	Go 1.22 + gRPC/TLS	Desplegado (pasd_nodo2)	Sincronización de tres vías; conflictos sin pérdida de datos; programada y al detectar la red local
MPI Jobs (RMI Server Java Cluster)	Java 17 + RMI + OpenMPI 5	Desplegado; integración de equipos externos sin probar	Código fuente y datasets; cola con envejecimiento; asignación por carga; trabajo k-means ejecutado en los 4 nodos con resultado en el Home
Monitoreo	Prometheus + Grafana	Desplegado (pasd_nodo1); nodos del clúster pendientes	17 objetivos de recolección, 12 reglas de alerta, tableros publicados en /monitoreo
Seguridad	CA interna, ufw, GnuPG	Parcial	TLS en todos los canales con credenciales; puertos internos cerrados; respaldos GPG sin llave del custodio
Clientes	Web, Python (Windows/Linux), Android	Integrados con las VPS	Los tres clientes consumen los servicios desplegados (sección 7.2)

La puesta en marcha se verificó en las tres VPS sobre las unidades de systemd. En pasd_nodo1 están activos slapd, apache2, nginx, upbc-node-web, prometheus, el exportador blackbox y grafana-server; en pasd_nodo2, postgresql, upbc-storage, upbc-shared-file y upbc-sync; en pasd_nodo3, postgresql, upbc-media, upbc-gestor-cluster y upbc-job-api. El cortafuegos ufw está activo en las tres máquinas.

7.2 Validación de extremo a extremo

La validación se ejecutó el 22 de octubre de 2026 contra el sistema desplegado, con un conjunto de pruebas organizado según las cuatro partes del objetivo de validación (Tabla 5). Las pruebas no se limitan a comprobar respuestas exitosas: contrastan lo que reporta el sistema con la realidad medida por otro camino.

- **Ejecución distribuida.** Un programa MPI en C, enviado por la API, informa en qué máquina corre cada proceso, hace circular un testigo por todos los procesos con MPI_Send/MPI_Recv y suma de forma repartida un conjunto de datos que todos leen por NFS; la prueba exige varias máquinas, el testigo completo y la suma exacta.
- **Clientes.** La interfaz web real se recorre en un navegador sin ventana y se registra cada petición de red, para comprobar que solo se habla con el portal. El cliente de escritorio se ejercita con su propio código, incluida la sincronización por gRPC/TLS. Del cliente móvil se verifican la configuración, la autoridad certificadora incluida en el APK, la validez de los certificados para Android y cada ruta declarada en sus interfaces Retrofit.

- **Telemetría.** N peticiones deben incrementar el contador de Prometheus exactamente en N; los bytes subidos deben reflejarse sin desviación en la métrica de espacio usado; la memoria y las CPU reportadas deben coincidir con las del sistema operativo, y las métricas y alertas del clúster con su estado real.
- **Seguridad.** Autenticación y mapa de servicios; ataques a las APIs (tokens falsificados o cerrados, identificador en el cuerpo, inyección SQL, enumeración de recursos ajenos), y el despliegue: TLS, versiones antiguas rechazadas, puertos internos cerrados desde fuera, cortafuegos activos y respaldos cifrados.

Tabla 5. Resultados de la validación de extremo a extremo (22 de octubre de 2026).

Parte del objetivo	Pruebas	Resultado	Observaciones
Ejecución distribuida de trabajos MPI	10	No concluyente en la ejecución automatizada	Las VMs del clúster estaban apagadas: la prueba lo detectó y omitió el envío. La ejecución distribuida se había verificado con un trabajo k-means en los 4 nodos (sección 7.3).
Integración de los clientes	42	42 correctas	Web (9), escritorio (19) y móvil (14)
Precisión de la telemetría	14	14 correctas	Contador +25 exacto, +123 457 bytes exactos, memoria y CPU idénticas, alertas coherentes con el clúster apagado
Esquemas de seguridad	63	57 correctas, 6 fallidas	Enumeración de recursos ajenos (4), JSON mal formado con error 500 (1), respaldos GPG inexistentes (1)

Las ejecuciones dejan además evidencia cuantitativa. En el cliente web, el recorrido completo generó 45 peticiones de red, 13 de ellas a /api y ninguna a un destino distinto del portal. En la telemetría, Prometheus tenía activos sus 17 objetivos de recolección; 25 peticiones reales incrementaron el contador http_peticiones_total exactamente en 25 y el histograma de latencia registró las mismas 25, con un percentil 95 de 9,9 ms para la consulta de cuota; la subida de un archivo incrementó la métrica de espacio usado exactamente en sus bytes, y la memoria total (2,89 GiB) y el número de CPU de cada VPS coincidieron con /proc/meminfo y nproc. En seguridad, las pruebas de autenticación y del mapa de servicios se superaron en su totalidad: el perfil estudiante recibe 403 en HPC y 200 en Storage, y un token cerrado deja de valer en todos los servicios.

La validación cumplió su propósito de contrastar la implementación con la especificación. Las seis observaciones de seguridad no exponen el contenido de ningún archivo ni permiten operar sobre recursos ajenos: en las cinco primeras el servicio rechaza la petición y la diferencia está solo en el código de respuesta. En cuatro de ellas, Storage responde 400 en lugar de 404 cuando un usuario ajeno usa el identificador de un archivo como carpeta, o al revés, porque comprueba el tipo del recurso antes que el permiso; invertir ese orden basta para que la respuesta no revele que el identificador existe. En la quinta, Storage y Photo Album responden 500 en lugar de 400 ante un cuerpo JSON mal formado, porque ese error de la capa de entrada no pasa por el traductor único de errores. La sexta corresponde a la operación y no al código: el cifrado de las copias de seguridad con GPG está implementado y su puesta en marcha depende de generar la llave del custodio. En la telemetría, las alertas «ejecutor desconectado» y «nodo en falla» estaban disparadas mientras el clúster permanecía apagado, lo que confirma que el monitoreo refleja el estado real de la infraestructura.

7.3 Rendimiento de la ejecución distribuida: k-means

El trabajo de prueba es un programa k-means en C con MPI (kmeans.c, publicado en RMIUPBCientifica/job-runner/ejemplos/ del repositorio del proyecto; véase la disponibilidad del código), que el servicio compila con mpicc y lanza con mpirun como cualquier otro trabajo. Su diseño prioriza que la comunicación no dependa del tamaño de los datos. El rango 0 lee el dataset y elige los centroides iniciales con k-means++ (Arthur y Vassilvitskii, 2007), con una semilla fija para que las ejecuciones sean reproducibles; MPI_Bcast difunde los centroides y MPI_Scatterv reparte los puntos en bloques que difieren como máximo en una fila. En cada iteración, cada proceso asigna solo sus puntos al centroide más cercano y acumula por grupo la suma de coordenadas, el número de puntos y el error cuadrático (Listado 1); dos reducciones MPI_Allreduce combinan esos acumulados, y todos los procesos calculan los mismos centroides nuevos sin que ninguno tenga que repartirlos. El cómputo por iteración es O(N·K·D/P), mientras que la comunicación se limita a K·D + K + 1 valores, independientemente del número de puntos N.

```
for (iteraciones = 1; iteraciones <= prm.max_iter; iteraciones++) {
    memset(sumas, 0, k * d * sizeof(double));  memset(conteo, 0, k * sizeof(long));
    double sse_local = 0;
    for (int i = 0; i < mis_filas; i++) {          /* solo los puntos propios: N/P */
        const double *x = mios + (size_t)i * d;
        int mejor = 0;  double dmejor = DBL_MAX;
        for (int c = 0; c < k; c++) {
            double t = distancia2(x, centroides + (size_t)c * d, d);
            if (t < dmejor) { dmejor = t; mejor = c; }
        }
        grupo[i] = mejor;  conteo[mejor]++;  sse_local += dmejor;
        for (int j = 0; j < d; j++) sumas[(size_t)mejor * d + j] += x[j];
    }
    /* sumas y SSE en un solo búfer: K·D + 1 valores, sin importar N */
    memcpy(buffer, sumas, k * d * sizeof(double));  buffer[k * d] = sse_local;
    MPI_Allreduce(MPI_IN_PLACE, buffer, k * d + 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
    MPI_Allreduce(conteo, conteo_global, k, MPI_LONG, MPI_SUM, MPI_COMM_WORLD);
    /* todos calculan los mismos centroides: buffer / conteo_global (sin difundirlos) */
    ...
    if (desplazamiento <= prm.tol) { convergio = 1; break; }
}
```

Listado 1. Núcleo iterativo de kmeans.c: cómputo local sobre N/P puntos y reducción global de los acumulados.

Tres decisiones del código reducen el costo por iteración: las sumas y el error cuadrático viajan en un único búfer reducido en el sitio (MPI_IN_PLACE), lo que ahorra un mensaje y una copia; los centroides se recalculan de forma redundante en todos los procesos, lo que sustituye una difusión por iteración por K·D divisiones locales, y el recorrido de cada proceso sobre sus puntos es secuencial en memoria contigua. El programa, además, mide por separado el bucle de cálculo y la ejecución completa, y permite omitir la escritura de asignaciones (asignaciones=0) para que la entrada y salida no altere las mediciones.

Las mediciones se realizaron con uno, dos, tres y cuatro nodos, y cada configuración se repitió varias veces. La Tabla 6 presenta el promedio y la mediana del tiempo de ejecución, la aceleración S(p) = T(1)/T(p) y la eficiencia E(p) = S(p)/p, y la Fig. 4 las representa frente a la aceleración ideal.

Tabla 6. Tiempos de ejecución de k-means en el clúster, aceleración y eficiencia.

Nodos	Promedio (s)	Mediana (s)	Aceleración	Eficiencia
1	1,1306	1,1385	1,00	100 %
2	0,7909	0,7903	1,43	71 %
3	0,7197	0,7181	1,57	52 %
4	0,6988	0,6991	1,62	40 %

El tiempo se reduce un 30 % al pasar de uno a dos nodos, pero cada nodo adicional aporta menos: de dos a cuatro nodos la reducción es de solo el 12 %, y la eficiencia cae del 71 % al 40 %. La mediana difiere del promedio en menos del 1 % en todas las configuraciones, de modo que la tendencia no se debe a mediciones aisladas sino al comportamiento del programa en el clúster.

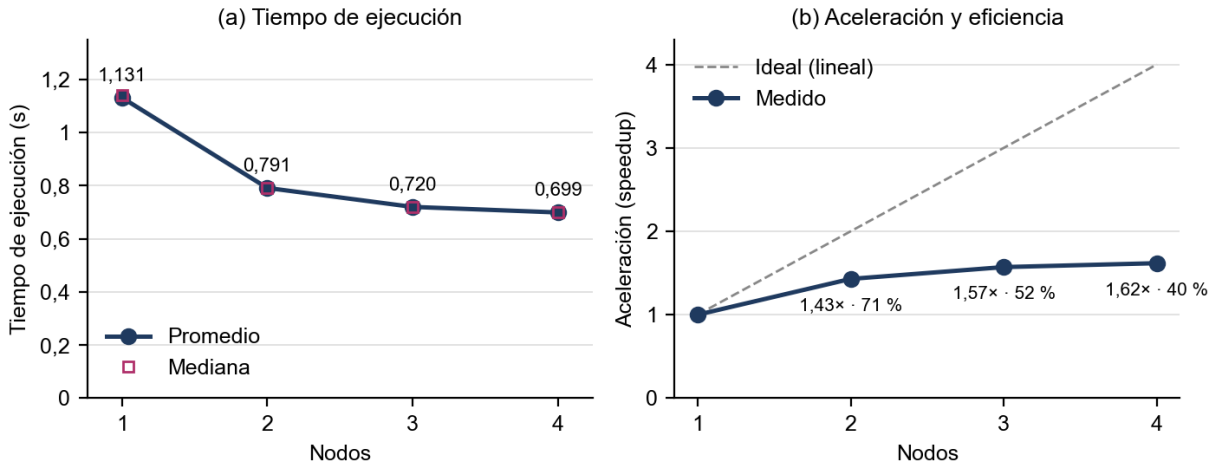


Fig. 4. Ejecución de k-means en el clúster: (a) tiempo de ejecución promedio y mediana; (b) aceleración medida frente a la ideal, con la eficiencia en cada punto.

La métrica de Karp y Flatt (1990), $e = (1/S - 1/p)/(1 - 1/p)$, estima la fracción no paralelizable a partir de la aceleración medida. Sus valores son 0,40, 0,45 y 0,49 para dos, tres y cuatro nodos. Si el límite fuera solo una porción secuencial fija del programa, como supone la ley de Amdahl, e se mantendría constante; que crezca con p indica que el costo dominante es la sobrecarga de la paralelización —comunicación y sincronización entre procesos—, que aumenta con cada nodo añadido.

El código permite identificar el origen de esa sobrecarga. Cada iteración termina en dos reducciones globales cuyo costo depende de la latencia de la red y crece con el número de procesos, mientras que el cómputo de cada proceso disminuye en proporción a 1/P. Con un tiempo secuencial cercano a un segundo, esa latencia —entre máquinas virtuales que comparten el hardware de los PC del laboratorio— es del mismo orden que el cómputo que se reparte. En la ejecución completa intervienen además fases secuenciales: el rango 0 lee o genera todos los puntos y ejecuta k-means++, y MPI_Scatterv envía por la red una fracción (P − 1)/P de los datos, que aumenta con cada nodo añadido. Se trata de escalabilidad fuerte, con el tamaño del problema fijo, por lo que estos costos pesan más a medida que crece P.

El análisis señala las optimizaciones con mayor efecto esperado. Fusionar las dos reducciones en una sola, enviando los conteos como valores de punto flotante en el mismo búfer, reduce a la mitad la latencia de comunicación por iteración. Que cada proceso lea su propio bloque del dataset desde el sistema de archivos compartido, por ejemplo con MPI-IO, elimina la lectura secuencial y la distribución desde el rango 0. La variante paralela k-means|| (Bahmani et al., 2012) reparte también la inicialización. Y, en la operación del servicio, asignar varios procesos por nodo y conjuntos de datos de mayor tamaño hace que el cómputo por proceso domine sobre la comunicación. La ejecución confirma que el servicio distribuye el trabajo en los cuatro nodos con una aceleración real de 1,62, y delimita con precisión dónde se pierde la eficiencia restante.

8 Cronograma

La Tabla 7 presenta el cronograma planificado para las 17 semanas del periodo académico 2026-20, calculado a partir del inicio de clases el 14 de julio de 2026.

Tabla 7. Cronograma del proyecto.

Sem.	Fechas 2026	Fase	Hito principal
1	14 – 20 jul	Planificación	Roles del equipo, alcance técnico y tecnologías validadas.
2	21 – 27 jul	Planificación / Diseño	Arquitectura general y contratos de API entre servicios.
3	28 jul – 3 ago	Diseño	Modelo de directorio activo, permisos Unix y esquema de seguridad.
4	4 – 10 ago	Iteración 1	Infraestructura Grid base y directorio activo desplegados.
5	11 – 17 ago	Iteración 1	Clúster HPC configurado; primeras pruebas de ejecución MPI.
6	18 – 24 ago	Iteración 2	Servicio File Sync (gRPC) funcional.
7	25 – 31 ago	Iteración 2	Repositorio distribuido y servicio Shared File funcionales.
8	1 – 7 sep	Cierre de iteración	Primer informe de seguimiento (bitácora).
9	8 – 14 sep	Iteración 3	Servicio Photo Album desplegado.
10	15 – 21 sep	Iteración 3	Servicio Streaming funcional.
11	22 – 28 sep	Iteración 4	Servicio MPI Jobs (Java RMI) integrado con el clúster.
12	29 sep – 5 oct	Iteración 4	Módulo de monitoreo desplegado.
13	6 – 12 oct	Integración	Integración completa y pruebas end-to-end.
14	13 – 19 oct	Producto	Entrega del producto funcional completo.
15	20 – 26 oct	Participación institucional	Evento obligatorio (20-22 oct.) y hardening de seguridad.
16	27 oct – 2 nov	Cierre	Segundo informe, artículo científico final y presentación JSPI.
17	3 – 9 nov	Sustentación	Sustentación de carácter individual.

Las desviaciones principales respecto al plan fueron dos. El despliegue en la infraestructura del CCA se realizó entre las semanas 14 y 15 y pasó de siete máquinas virtuales previstas a tres servidores virtuales, lo que obligó a agrupar los servicios sin modificarlos. La ejecución de trabajos MPI en el clúster real, prevista para la semana 11, se completó también en la semana 15, después de resolver la configuración de red de las máquinas virtuales del laboratorio. Hasta entonces, el servicio se había validado con un ejecutor MPI simulado.

9 Discusión y conclusiones

El reto central del proyecto fue integrar servicios desarrollados por subequipos distintos, en tecnologías que no pueden compartir código, y desplegarlos sobre una infraestructura distinta de la planificada. De la experiencia se desprenden las siguientes conclusiones.

En un sistema heterogéneo, el contrato es la arquitectura. Fijar los contratos antes de implementar, y validar cada servicio contra un validador de tokens intercambiable, permitió desarrollar y probar cada componente antes de que existiera su dependencia real, e integrar sin rehacer interfaces. El costo es la

duplicación deliberada de piezas comunes, como el validador de tokens, que deben mantenerse alineadas, y el efecto en cadena de cualquier cambio de contrato.

Autenticar en un solo lugar no significa autorizar en un solo lugar. La identidad se centraliza en el servicio SOAP, pero cada servicio decide el acceso en cada petición, y la pasarela no toma decisiones: saltarla no concede ningún acceso. La caché de validación en los servicios reduce la latencia a cambio de una ventana acotada en la que un token revocado todavía se acepta, un compromiso explícito entre rendimiento y rapidez de revocación.

Una sola fuente de verdad para los datos simplifica la consistencia a cambio de dependencia en tiempo de ejecución. Que Storage sea el único dueño de los bytes y de los permisos eliminó la sincronización de copias y la consistencia eventual entre servicios; los demás guardan solo índices reconstruibles. La disponibilidad de Storage acota, en contrapartida, la de casi todo el sistema.

Separar el ciclo de vida de la petición del ciclo de vida del trabajo. Las operaciones costosas —la ejecución de trabajos MPI, la indexación de medios, la sincronización— se modelaron con colas y máquinas de estados explícitas, lo que hizo posibles los reintentos, la reasignación ante fallos y la consulta del progreso sin bloquear al usuario.

En un sistema distribuido, la red es parte del diseño. Conservar los nombres lógicos como alias permitió consolidar siete nodos en tres servidores sin tocar el código. El clúster real confirmó la lección: la mayoría de las fallas no fueron de código sino de configuración de red —máquinas con varias direcciones, interfaces elegidas incorrectamente por OpenMPI, direcciones anunciadas por RMI y puertos dinámicos bloqueados—, y se resolvieron declarando explícitamente interfaces, nombres y puertos.

Un entorno simulado valida contratos; solo el entorno real valida el sistema. El servicio HPC funcionaba de punta a punta con un ejecutor simulado y, aun así, el clúster real expuso problemas que la simulación no podía mostrar. Del mismo modo, las pruebas de extremo a extremo contra el despliegue revelaron defectos de seguridad que las pruebas funcionales no detectaban, y la observabilidad incorporada desde el despliegue permitió distinguir fallas de aplicación de fallas de infraestructura con datos y no con suposiciones.

Como contribución, este trabajo documenta un caso aplicado de integración de múltiples paradigmas de sistemas distribuidos —objetos distribuidos, servicios web, RPC binario, paso de mensajes y computación en malla— en una solución coherente y verificable, que puede servir de referencia para futuras infraestructuras científicas en la UPB seccional Bucaramanga.

Disponibilidad del código. El código fuente del sistema está publicado en la organización https://github.com/UPBCIENTIFICA-2026-02: los servicios, las pruebas de extremo a extremo y el programa k-means en el repositorio UPB-CIENTIFICA; los clientes en FrontWebUPBCientifica (web), FrontDesktopUPBCientifica (escritorio) y FrontMovilUPBCientifica (Android).

Agradecimientos. Los autores agradecen al docente MSc. Lenin Javier Serrano Gil por la orientación brindada durante el desarrollo de este proyecto, así como a la Facultad de Ingeniería de Sistemas e Informática de la UPB seccional Bucaramanga por el acceso al Centro de Computación Avanzada.

Conflicto de intereses. Los autores declaran no tener conflictos de interés relacionados con el contenido de este artículo.

Referencias bibliográficas

1. Adepoju, S., & Ologunagba, G. F. (2025). A systematic review and comparative evolution of grid computing and modern cloud computing infrastructures. International Journal of Scientific Research in Computer Science and Engineering, 13(5), 43–51.
2. AlHafez, N., & Kurdi, A. (2025). Parallel paradigms in modern HPC: A comparative analysis of MPI, OpenMP, and CUDA. arXiv preprint arXiv:2506.15454.
3. Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 1027–1035.
4. Babu, R., Abraham, G., & Borasia, K. (2013). A review on securing distributed systems using symmetric key cryptography. arXiv preprint arXiv:1303.0351.
5. Bahmani, B., Moseley, B., Vattani, A., Kumar, R., & Vassilvitskii, S. (2012). Scalable k-means++. Proceedings of the VLDB Endowment, 5(7), 622–633.
6. Capra, R. (2014). File synchronization and sharing: User practices and challenges. Proceedings of the American Society for Information Science and Technology, 51(1), 1–10.
7. Castillo, P. A., Bernier, J. L., Arenas, M. G., Merelo, J. J., & García-Sánchez, P. (2011). SOAP vs REST: Comparing a master-slave GA implementation. arXiv preprint arXiv:1105.4978.
8. Guerraoui, R., & Schiper, A. (1996). Fault-tolerance by replication in distributed systems. En A. Strohmeier (Ed.), Reliable Software Technologies — Ada-Europe ’96 (LNCS, vol. 1088, pp. 38–57). Springer.
9. Juric, M. B. (2000). Comparison of CORBA and Java RMI based on performance analysis. Actas de los Workshops de ECOOP.
10. Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. Communications of the ACM, 33(5), 539–543.
11. Kunz, P. (2022). HPC job-monitoring with SLURM, Prometheus and Grafana (Tesis de pregrado). Universidad de Basilea.
12. Mehdi, Z., & Ragab-Hassen, H. (2016). File synchronization systems survey. Actas de ICCSEA, SPPR, UBIC 2016, 29–38.
13. Munoz, C., & Zalewski, J. (2001). Architecture and performance of Java-based distributed object models: CORBA vs RMI. Real-Time Systems, 21(1).
14. Natali, L., & Merani, M. L. (2015). Adaptive streaming in P2P live video systems: A distributed rate control approach. arXiv preprint arXiv:1510.04472.
15. Priyadarshini, S. B. B. (2022). Security in distributed system: A review perspective.
16. Sommerville, I. (2011). Software Engineering (9.ª ed.). Addison-Wesley.
17. Sungkar, A., & Kogoya, T. (2020). A review of grid computing. Computer Science & IT Research Journal, 1(1), 1–6.
18. UPB. (2020). Carta descriptiva del curso: Sistemas distribuidos. Código DA-FO-085N, versión 3_3.
19. Waheed, M. H. B., Jamil, F., Qayyum, A., Jamil, H., Cheikhrouhou, O., Ibrahim, M., Bhushan, B., & Hmam, H. (2021). A new efficient architecture for adaptive bit-rate video streaming. Sustainability, 13(8), 4541.
20. Woo, T. Y. C., & Lam, S. S. (1992). Authentication for distributed systems. IEEE Computer, 25(1), 39–51.